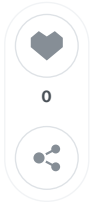


토큰화와 임베딩

goblurry · 방금 전

통계 수정 삭제



5. 토큰화

1. 단어 및 글자 토큰화
2. 형태소 토큰화
3. 하위 단어 토큰화
4. 토큰라이저스 라이브러리

개요

자연어(Natural Language)는 인공적으로 만들어진 프로그래밍 언어와 달리, 사람들이 자연히 만들어 쓰는 언어를 의미한다. **자연어 처리(Natural Language Processing, NLP)**는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술을 의미한다.

자연어 처리는 인공지능의 하위 분야 중 하나로, 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘과 모델을 개발한다. 이러한 모델을 개발하기 위해서는 다음과 같은 문제가 해결되어야 한다.

- **모호성(Ambiguity)**: 단어와 구가 사용되는 맥락에 따라 여러 의미를 가질 수 있어 명확히 구분할 수 있어야 한다.
- **기변성(Variability)**: 사투리, 강세, 신조어, 작문 스타일 등 다양한 가변성을 처리할 수 있어야 한다.
- **구조(Structure)**: 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 한다.

위와 같은 문제를 다루려면 우선 말뭉치(Corpus)를 일정한 단위인 **토큰(Token)**으로 나눠야 한다. 이 과정을 **토큰화(Tokenization)**라고 한다.

토큰화를 위해 **토큰라이저(Tokenizer)**를 사용한다. 토큰라이저란 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어를 의미한다. 토큰라이저를 구축하는 방법은 다음과 같다.

- 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
- 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
- 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
- 머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다.

이 방법 중 **어휘 사전(Vocabulary, Vocab)** 방식은 사전에 없는 단어 토큰인 **OOV(Out of Vocab)** 문제가 발생할 수 있다. OOV 문제를 고려해 토큰화하는 것이 중요하다.

1. 단어 및 글자 토큰화

텍스트 데이터를 단어(Word)나 글자(Character) 단위로 나누는 기법으로 각각 **단어 토큰화**와 **글자 토큰화**가

5. 토큰화

개요

1. 단어 및 글자 토큰화
 - 단어 토큰화
 - 글자 토큰화
2. 형태소 토큰화
 - KoNLPy
 - NLTK
 - spaCy
3. 하위 단어 토큰화
 - 바이트 페어
 - 워드피스
 - 센텐스피스
4. 토큰라이저

6. 임베딩

개요

1. 텍스트 벡터
 - 원-핫 인코딩
 - 빈도 벡터
 - TF-IDF
2. 언어 모델
 - N-gram
3. Word2Vec
 - CBOW
 - Skip-gram

있다. 이러한 기법을 통해 각각의 토큰은 의미를 갖는 최소 단위로 분해된다.

단어 토큰화

단어 토큰화(Word Tokenization) 는 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나로, 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업이다.

대부분의 언어는 띄어쓰기를 이용해 문장을 의미 있는 단위로 나눠 표현한다. 파이썬 문자열의 `split()` 메서드를 사용해 가장 간단하게 단어 토큰화를 수행할 수 있다.

```
# 예제 5.1 단어 토큰화
sentence = "형태소 분석기를 이용해 간단하게 토큰화할 수 있다."
tokenized = sentence.split()
print(tokenized)
```

출력 결과

```
['형태소', '분석기를', '이용해', '간단하게', '토큰화할', '수', '있다.']
```

단어 토큰화는 입력 시퀀스의 길이가 짧다는 장점이 있지만, '분석기를'처럼 조사가 붙은 형태를 하나의 토큰으로 처리하거나 어휘 사전에 없는 OOV 문제가 발생할 수 있다는 단점이 있다.

글자 토큰화

글자 토큰화(Character Tokenization) 는 텍스트를 개별 글자 단위로 나누는 방식이다. 언어 모델링과 같은 시퀀스 예측 작업에서 활용된다.

```
# 예제 5.2 글자 토큰화
review = "현실과 구분 불가능한 cg."
tokenized = list(review)
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.']
```

단어 토큰화와 달리 공백도 토큰으로 나뉜다. 한국어의 경우 하나의 글자는 여러 자음과 모음의 조합으로 이루어져 있다. 따라서 **자소(字素)**² 단위로 나뉘서 자소 단위 토큰화를 수행할 수도 있으며, 이 책에서는 `jamo` 라이브러리를 활용한다.

2. 형태소 토큰화

형태소 토큰화(Morpheme Tokenization) 란 텍스트를 형태소 단위로 나누는 토큰화 방법으로, 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업이다.

형태소 토큰화는 한국어와 같이 **교착어(Agglutinative Language)** 인 언어에서 특히 중요하게 수행된다. 한국어는 어근에 다양한 접사와 조사가 결합되어 하나의 낱말을 이루므로, 각 형태소를 적절히 구분해 처리해야 한다.

형태소는 크게 두 가지로 구분된다.

- **자립 형태소(Free Morpheme):** 스스로 의미를 가지는 것. 명사, 동사, 형용사와 같은 단어를 이루는 기본 단위.
- **의존 형태소(Bound Morpheme):** 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 것.

조사, 어미, 접두사, 접미사 등.

KoNLPy

KoNLPy는 한국어 자연어 처리를 위해 개발된 파이썬 라이브러리이다. Okt, Kkma, Komoran, Hannanum, Mecab 등 다양한 형태소 분석기를 제공한다.

```
# 예제 5.4 Okt 토큰화
from konlpy.tag import Okt

okt = Okt()
sentence = "형태소 분석기를 이용해 간단하게 토큰화할 수 있다."

print("명사 추출 :", okt.nouns(sentence))
print("구문 추출 :", okt.phrases(sentence))
print("형태소 추출 :", okt.morphs(sentence))
print("품사 태깅 :", okt.pos(sentence))
```

출력 결과

```
명사 추출 : ['형태소', '분석기']
구문 추출 : ['형태소 분석기']
형태소 추출 : ['형태소', '분석기', '를', '이용해', '간단하게', '토큰화', '할', '수', '있다', '.']
품사 태깅 : [('형태소', 'Noun'), ('분석기', 'Noun'), ('를', 'Josa'), ('이용해', 'Verb'), ...]
```

Okt 객체에서 지원하는 대표적인 메서드는 **명사 추출(okt.nouns)**, **구문 추출(okt.phrases)**, **형태소 추출(okt.morphs)**, **품사 태깅(okt.pos)** 이다.

Kkma 토큰라이저도 유사한 방식으로 사용하며, 형태소 분석 정확도가 높다는 특징이 있다.

NLTK

NLTK(Natural Language Toolkit)는 자연어 처리를 위해 개발된 라이브러리로, 토큰화, 형태소 분석, 구문 분석, 개체명 인식⁴, 감성 분석 등의 기능을 제공한다. 주로 영어 자연어 처리를 위해 개발됐지만, 네덜란드어, 프랑스어, 독일어 등 다양한 언어도 지원한다.

```
pip install nltk
```

이 책에서는 **Punkt 모델**과 **Averaged Perceptron Tagger** 모델을 활용해 토큰화 및 품사 태깅 작업을 수행한다.

spaCy

spaCy는 GPU 가속을 비롯해 영어, 프랑스어, 한국어, 일본어 등 24개 이상의 언어로 사전 학습된 모델을 제공한다. 이 책에서는 영어 대상 토큰화를 위해 영어로 사전 학습된 `en_core_web_sm` 모델을 설치한다.

```
# 예제 5.9 spaCy 품사 태깅
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"{token.pos_:5} - {token.tag_:3} : {token.text}")
```

출력 결과

```
[PRON - DT ] : Those
```

```
[PRON - WP ] : who
[AUX - MD ] : can
[VERB - VB ] : imagine
...
```

spaCy는 사전 학습된 모델을 기반으로 처리하기 때문에 spaCy 모델 불러오기 함수(load) 를 통해 모델을 설정한다. spaCy는 객체 지향적(Object Oriented) 으로 구현돼 처리한 결과를 doc 객체에 저장하며, 이 token 객체에 대한 정보를 기반으로 다양한 자연어 처리 작업을 수행한다.

3. 하위 단어 토큰화

자연어 처리에서 형태소 분석은 중요한 전처리 과정 중 하나이다. 그러나 신조어나 고유어, 오타자 등이 있는 문장을 기존 형태소 분석기로 토큰화하면 OOV 문제가 발생하기 쉽다.

이에 대한 해결책이 하위 단어 토큰화(Subword Tokenization) 이다. 하위 단어 토큰화 방법으로는 바이트 페어 인코딩, 워드피스, 유니그램 모델 등이 있다.

바이트 페어 인코딩

- **바이트 페어 인코딩(Byte Pair Encoding, BPE)** 이란 다이그램 코딩(Digram Coding)이라고도 하며 하위 단어 토큰화의 한 종류이다. 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘으로 초기에는 데이터 압축을 위해 개발됐으나, 자연어 처리 분야에서 하위 단어 토큰화를 위한 방법으로 사용된다.

이 알고리즘은 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하며, 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다. 가령 'abracadabra'라는 단어를 바이트 페어 인코딩하면 다음과 같이 처리된다.

- 원문: abracadabra
- Step #1: AracadAra (ab → A)
- Step #2: ABcadAB (ra → B)
- Step #3: CcadC (AB → C)

바이트 페어 인코딩은 입력 데이터에서 가장 많이 등장한 글자의 빈도수를 측정하고, 가장 빈도수가 높은 글자 쌍을 탐색한다. 이 방법을 반복해 최종적으로 어휘 사전을 구축한다.

워드피스

워드피스(WordPiece) 는 BPE와 유사하지만, 빈도수 대신 점수(score) 를 기준으로 병합할 글자 쌍을 선택한다.

$$\text{score} = f(x, y) / (f(x) \cdot f(y))$$

f는 빈도(frequency)를 나타내는 함수이며, x와 y는 병합하려는 하위 단어를 의미한다. 따라서 f(x, y)는 x와 y가 조합된 글자 쌍의 빈도, 즉 xy 글자 쌍의 빈도가 된다. 그러므로 score는 x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수가 된다.

이렇게 선택된 하위 단어는 이후에 더 높은 확률로 선택될 가능성이 높으며, 이를 통해 모델이 좀 더 정확한 하위 단어로 분리할 수 있다.

센텐스피스

센텐스피스(SentencePiece) 는 BPE와 유니그램 모델을 지원하는 라이브러리로, 공백을 특수 기호 `_` 로 치환하여 언어에 종속되지 않는 토큰라이저를 만들 수 있다.

```
# 예제 5.12 토큰라이저 모델 학습
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=../datasets/corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 model_type=bpe"
)
```

센텐스피스 라이브러리는 `SentencePieceTrainer` 클래스의 `Train` 메서드로 토큰라이저 모델을 학습할 수 있다.

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰라이저가 다룰 수 있는 글자의 비율

센텐스피스 토큰라이저 모델은 `SentencePieceProcessor` 클래스를 통해 학습된 모델을 불러올 수 있다. `encode_as_pieces` 메서드는 문장을 토큰화하며, `encode_as_ids` 메서드는 토큰을 정수로 인코딩해 제공한다. 반대로 `decode_ids` 나 `decode_pieces` 메서드를 통해 다시 문자열 데이터로 변환할 수 있다.

4. 토큰라이저스 라이브러리

토큰라이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토큰라이저를 구현하고 학습할 수 있다. 이번 실습에서는 센텐스피스 라이브러리 대신 허깅 페이스의 **토큰라이저스(Tokenizers)** 라이브러리를 사용한다.

```
pip install tokenizers
```

토큰라이저스 라이브러리는 **정규화(Normalization)** 와 **사전 토큰화(Pre-tokenization)** 를 제공한다.

정규화는 일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거하는 등의 작업을 수행한다. 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등을 제공한다.

사전 토큰화는 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능을 제공한다. 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효율적으로 처리하고 모델의 성능을 향상시킬 수 있다.

```
# 예제 5.15 워드피스 토큰라이저 학습
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())
```

워드피스 토큰라이저는 바이트 페어 인코딩 토큰라이저와 마찬가지로 위 과정을 반복해 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습한다.

6. 임베딩

1. 텍스트 벡터화
2. 언어 모델
3. Word2Vec
4. fastText
5. 순환 신경망 (RNN)
6. 합성곱 신경망 (CNN)

개요

앞서 입력된 문장을 컴퓨터가 이해할 수 있는 단위인 토큰으로 나누는 토큰화 과정에 대해 알아봤다. 그러나 토큰화만으로는 모델을 학습할 수 없다. 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 **텍스트 벡터화(Text Vectorization)** 과정이 필요하다.

기초적인 텍스트 벡터화로는 **원-핫 인코딩(One-Hot Encoding)**, **빈도 벡터화(Count Vectorization)** 등이 있다.

이러한 방법은 단어나 문장을 벡터 형태로 변환하기 쉽고 간단하다는 장점이 있지만, 벡터의 **희소성(Sparsity)** 이 크다는 단점이 있다. 또한, 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않으므로 두 문장이 의미적으로 유사하다고 해도 벡터가 유사하게 나타나지 않을 수 있다.

이러한 문제를 해결하기 위해 **워드 투 벡터(Word2Vec)** 나 **패스트 텍스트(fastText)** 등과 같이 단어의 의미를 학습해 표현하는 **워드 임베딩(Word Embedding)** 기법을 사용한다.

워드 임베딩은 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다는 단점이 있어 인공 신경망을 활용해 **동적 임베딩(Dynamic Embedding)** 기법을 사용하기도 한다. 이번 장에서는 기초적인 임베딩 기법부터 동적 임베딩 기법까지 알아본다.

1. 텍스트 벡터화

원-핫 인코딩

원-핫 인코딩(One-Hot Encoding) 이란 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식이다.

예를 들어, 'I like apples'와 'I like bananas' 문장을 원-핫 인코딩하면 다음과 같다.

색인	0	1	2	3
토큰	I	like	apples	bananas

- I like apples: [1, 1, 1, 0]
- I like bananas: [1, 1, 0, 1]

빈도 벡터화

빈도 벡터화(Count Vectorization) 는 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식이다. 원-핫 인코딩 방식은 같은 단어가 여러 번 등장하더라도 1과 0으로만 표현하지만, 빈도 벡터화는 해당 단어의 빈도로 표시한다.

TF-IDF

원-핫 인코딩과 TF-IDF 같은 방법은 벡터 요소의 대부분이 0으로 표현되는 희소 표현 방법이다. 또한 단어 간의 유사성을 반영하지 못하고 벡터 간의 유사성을 계산하는 데도 많은 비용이 발생한다.

TF-IDF(Term Frequency-Inverse Document Frequency) 는 문서 빈도와 역문서 빈도를 곱한 값으로, 특정 단어가 문서에 자주 등장하지만 전체 문서에서는 드물게 등장할수록 값이 커진다. 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아진다.

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

파이썬의 **사이킷런(Scikit-learn)** 라이브러리를 활용해 TF-IDF를 계산할 수 있다.

2. 언어 모델

언어 모델(Language Model) 은 단어 시퀀스에 확률을 부여하는 모델이다. 이전에 등장한 단어들을 바탕으로 다음 단어가 등장할 확률을 계산한다.

언어 모델에서 조건부 확률은 연쇄법칙을 이용해 계산된다. 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타낼 수 있다.

$$P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2|w_1) \dots P(w_n|w_1, w_2, \dots, w_{n-1})$$

최근 연구되는 자연어 처리 기법은 언어 모델을 활용해 가중치를 사전 학습한다. 예를 들어 GPT(Generative Pre-trained Transformer)나 BERT(Bidirectional Encoder Representations from Transformers)는 다음과 같은 문장 생성 기법을 이용해 모델을 사전 학습한다.

- **GPT**: Raw Sentence 방식으로 다음 단어를 예측한다.
- **BERT**: 마스킹된 단어([MASK])를 예측한다.

N-gram

가장 기초적인 통계적 언어 모델은 **N-gram** 모델이다. N-gram 모델은 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.

통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성하므로, 이를 이용하여 새로운 문장을 생성할 수 있으며, 다양한 종류의 텍스트 데이터를 학습할 수 있다.

3. Word2Vec

Word2Vec 은 밀집 표현으로 단어를 고정된 크기의 실수 벡터로 표현하기 때문에 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않는다. 벡터 공간에서 단어 간의 거리를 효과적으로 계산할 수 있으며, 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적으로 공간을 활용할 수 있다.

밀집 표현 벡터화는 학습을 통해 단어를 벡터화하기 때문에 단어의 의미를 비교할 수 있다. 밀집 표현된 벡터를 단어 **임베딩 벡터(Word Embedding Vector)** 라고 하며, Word2Vec은 대표적인 단어 임베딩 기법 중 하나이다.

Word2Vec은 밀집 표현을 위해 **CBow** 와 **Skip-gram** 이라는 두 가지 방법을 사용한다.

CBoW

CBoW(Continuous Bag of Words)란 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법이다. **중심 단어(Center Word)**는 예측해야 할 단어를 의미하며, 예측에 사용되는 단어들을 **주변 단어(Context Word)**라고 한다.

중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지를 정해야 하는데, 이 범위를 **윈도우(Window)**라고 한다. CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습할 수 있다.

```
# 예제 6.5 Skip-gram의 단어 쌍 추출
def get_word_pairs(tokens, window_size):
    pairs = []
    for sentence in tokens:
        sentence_length = len(sentence)
        for idx, center_word in enumerate(sentence):
            window_start = max(0, idx - window_size)
            window_end = min(sentence_length, idx + window_size + 1)
            context_words = sentence[window_start:idx] + sentence[idx+1:window_end]
            for context_word in context_words:
                pairs.append([center_word, context_word])
    return pairs
```

Skip-gram

Skip-gram은 CBoW와 반대로 중심 단어를 가지고 주변 단어를 예측하는 방법이다. CBoW에 비해 성능이 더 좋은 것으로 알려져 있다.

네거티브 샘플링

네거티브 샘플링(Negative Sampling)은 Word2Vec 모델에서 사용되는 확률적인 샘플링 기법으로, 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용한다.

학습 윈도우 내에 등장하지 않는 단어를 n 개 추출하여 정답 단어와 함께 소프트맥스 연산을 수행한다. 이를 통해 전체 단어의 확률을 계산할 필요 없이 모델을 효율적으로 학습할 수 있다. 이때 추출할 단어 n 은 일반적으로 5~20개를 사용한다.

각 단어 w_i 가 네거티브 샘플로 추출될 확률 $P(w_i)$ 는 출현 빈도수에 0.75제곱한 값을 정규화 상수로 사용하는데, 이 값은 실험을 통해 얻어진 최적의 값이다.

단어 유사도 계산

단어 임베딩 벡터 간의 유사도는 **코사인 유사도(Cosine Similarity)**로 측정한다. 코사인 유사도는 두 벡터를 내적인 벡터의 크기의 곱을 나눠 계산한다.

```
# 예제 6.12 단어 임베딩 유사도 계산
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b):
    cosine = np.dot(b, a) / (norm(b, axis=1) * norm(a))
    return cosine

def top_n_index(cosine_matrix, n):
    closest_indexes = cosine_matrix.argsort()[::-1]
    top_n = closest_indexes[1 : n + 1]
    return top_n

cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n=5)
```



```
print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도 : {cosine_matrix[index]:.4f}")
```

출력 결과

```
연기와 가장 유사한 5 개 단어
연기력 - 유사도 : 0.3976
배우 - 유사도 : 0.3167
시나리오 - 유사도 : 0.3130
악마 - 유사도 : 0.2977
까지도 - 유사도 : 0.2892
```

4. fastText

fastText²는 2015년 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델로, 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘이다. fastText는 단어와 문장을 벡터로 변환하는 기술을 기반으로 하며, 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용된다.

이러한 벡터화 기술은 Word2Vec과 유사하지만, fastText는 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능을 제공한다. 하위 단어를 고려하기 위해 **N-gram**을 사용해 단어를 분해하고 벡터화하는 방법으로 동작한다.

fastText에서는 단어의 벡터화를 위해 < , > 와 같은 특수 기호를 사용하여 단어의 시작과 끝을 나타낸다. 이러한 기호가 추가된 단어는 N-gram을 사용하여 **하위 단어 집합(Subword set)**으로 분해된다. 예를 들어 '서울특별시'를 바이그램으로 분해하면 '서울', '울특', '특별', '별시'가 된다.

OOV 처리

Word2Vec 모델과 달리 fastText는 OOV 단어를 대상으로도 의미 있는 임베딩을 추출할 수 있다.

```
# 예제 6.17 fastText OOV 처리
oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn=5))
```

출력 결과

```
False
[('사랑', 0.8812...), ('사랑에', 0.8438...), ('사랑의', 0.7931...), ('사랑을', 0.7594...), ('사랑
```

'사랑해요'라는 단어는 어휘 사전 내에 존재하지 않으므로 OOV 토큰으로 처리된다. '사랑해요' 토큰은 '사랑', '랑해', '해요' 등의 하위 단어로 분해되며, 이 분해된 하위 단어의 임베딩을 모두 합해 전체 토큰의 임베딩을 계산한다. 이와 같은 방식으로 fastText는 OOV 문제를 효과적으로 해결할 수 있다.

5. 순환 신경망 (RNN)

순환 신경망(Recurrent Neural Network, RNN) 모델은 순서가 있는 연속적인 데이터(Sequence data)를 처리하는 데 적합한 구조를 갖고 있다. 순환 신경망은 각 시점(Time step)의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동한다.

순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을

예측하므로 연속형 데이터를 처리할 수 있다.

순환 신경망은 다양한 구조로 모델을 설계할 수 있다. 가장 단순한 구조인 단순 순환 구조부터 일대다 구조, 다대일 구조, 다대다 구조 등이 있다.

일대다 구조

일대다 구조(One-to-Many) 는 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조다. 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 **이미지 캡셔닝(Image Captioning)** 모델이 대표적인 예이다.

다대일 구조

다대일 구조(Many-to-One) 는 여러 개의 입력 시퀀스를 받아 하나의 출력값을 생성하는 순환 신경망 구조다. 텍스트 감성 분류나 문서 분류 작업에 활용된다.

다대다 구조

다대다 구조(Many-to-Many) 는 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조다. 기계 번역, 음성 인식 시스템 등에서 사용된다.

다대다 구조는 **시퀀스-시퀀스(Seq2Seq)** 구조로 이뤄져 있다. 시퀀스-시퀀스 구조는 입력 시퀀스를 처리하는 **인코더(Encoder)** 와 출력 시퀀스를 생성하는 **디코더(Decoder)** 로 구성된다. 인코더는 입력 시퀀스를 처리해 고정 크기의 벡터를 출력하고, 디코더는 이 벡터를 입력으로 받아 출력 시퀀스를 생성한다.

양방향 순환 신경망

양방향 순환 신경망(Bidirectional Recurrent Neural Network, BiRNN) 은 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식이다.

순환 신경망은 현재 시점의 입력값을 처리하기 위해 이전 시점($t-1$)의 은닉 상태만을 이용하는데, 양방향 순환 신경망에서는 이전 시점($t-1$)의 은닉 상태뿐만 아니라 이후 시점($t+1$)의 은닉 상태도 함께 이용한다.

장단기 메모리 (LSTM)

일반적인 순환 신경망은 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값을 예측하는 구조이므로 시간적으로 먼 과거의 정보는 잘 기억하지 못한다. 이러한 단점으로 인해 **장기 의존성 문제(Long-term dependencies)**가 발생할 수 있다. 또한, 활성화 함수로 사용되는 하이퍼볼릭 탄젠트 함수나 ReLU 함수의 특성으로 인해 역전파 과정에서 기울기 소실이나 기울기 폭주가 발생할 가능성이 있다.

이러한 문제를 해결하기 위해 **장단기 메모리(Long Short-Term Memory, LSTM)** 를 사용한다. 장단기 메모리는 순환 신경망과 비슷한 구조를 가지지만, **메모리 셀(Memory cell)** 과 **게이트(Gate)** 라는 구조를 도입해 장기 의존성 문제와 기울기 소실 문제를 해결한다.

장단기 메모리는 셀 상태(Cell state)와 세 가지 게이트로 정보의 흐름을 제어한다.

- **망각 게이트(Forget gate):** 이전 시점의 메모리 셀과 현재 시점의 입력을 바탕으로 어떤 정보를 삭제할지 결정한다. 출력값이 1에 가까우면 더 많은 정보를 유지하고, 0에 가까우면 더 많은 정보를 삭제한다.
- **기억 게이트(Input gate):** 현재 시점의 새로운 정보를 얼마나 받아들일지 결정한다.
- **출력 게이트(Output gate):** 현재 시점의 은닉 상태를 제어한다.

```
# 예제 6.20 문장 분류 모델 (RNN/LSTM 선택)
from torch import nn
```

```

class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm",
    ):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )
        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )
        elif model_type == "lstm":
            self.model = nn.LSTM(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

```

사전 학습된 임베딩 적용

Word2Vec 등으로 학습된 임베딩을 모델의 임베딩 계층에 적용할 수 있다. 사전 학습된 임베딩을 적용하면 모델이 더 빠르게 수렴하고 성능이 향상될 수 있다.

```

# 예제 6.29 사전 학습된 임베딩 계층 적용
if pretrained_embedding is not None:
    self.embedding = nn.Embedding.from_pretrained(
        torch.tensor(pretrained_embedding, dtype=torch.float32)
    )
else:
    self.embedding = nn.Embedding(
        num_embeddings=n_vocab,
        embedding_dim=embedding_dim,
        padding_idx=0
    )

```

<pad> 토큰과 <unk> 토큰은 초기화에서 제외한다. 임베딩 계층은 파이토치의 임베딩 클래스의 `from_pretrained` 메서드로 초기화할 수 있다.

6. 합성곱 신경망 (CNN)

합성곱 신경망(Convolutional Neural Network, CNN)은 이미지와 유사한 구조를 가진 2차원 데이터에 더 적합하므로 텍스트 데이터를 합성곱 신경망으로 처리한다면 모델의 구조 및 목적을 충분히 고려해야 한다.

합성곱 계층은 필터를 사용해 데이터의 특징을 추출하므로 데이터의 지역적인 패턴을 인식할 수 있으며, 입력 데이터의 모든 위치에서 동일한 필터를 사용하므로 모델 매개변수를 공유한다. 이러한 합성곱 계층을 여러 겹

쌓아 모델을 구성하며, 합성곱 계층이 많아질수록 모델의 복잡도가 증가하므로 더 다양한 특징을 추출해 학습할 수 있다.

필터

합성곱 계층은 입력 데이터에 **필터(Filter)** 를 이용해 합성곱 연산을 수행하는 계층이다.

필터는 **커널(Kernel)** 또는 **윈도우(Window)** 로 불리기도 하며, 일반적으로 3×3, 5×5와 같은 작은 크기의 정방형으로 구성된다.

이 필터를 일정 간격으로 이동하면서 입력 데이터와 합성곱 연산을 수행해 특징 맵을 생성한다. 합성곱 신경망은 보통 여러 개의 필터를 사용해 다양한 특징을 추출하며, 이를 통해 입력 이미지의 다양한 특징을 인식하고 분류할 수 있다.

패딩

패딩(Padding) 은 입력 데이터의 가장자리에 특정 값을 추가하는 것이다. **제로 패딩(Zero Padding)** 은 입력 이미지의 각각의 픽셀 주위에 0을 추가한 뒤 합성곱 연산을 수행하며, 패딩을 추가하면 출력값의 크기가 작아지지 않고 입력값 크기와 동일하게 계산된다.

간격

간격(Stride) 이란 필터가 한 번에 움직이는 크기를 의미한다. 간격의 크기를 조절함으로써 출력 데이터의 크기를 조절할 수 있다. 간격을 조정함으로써 입력 데이터의 공간적인 정보를 유지하거나 감소시킬 수 있다.

풀링

풀링(Pooling) 은 특징 맵의 크기를 줄이는 연산으로 합성곱 계층 다음에 적용된다. 풀링은 특징 맵의 크기를 줄여 연산량을 감소시키고 입력 데이터의 정보를 압축하는 효과를 가진다.

선택하는 방법에는 크게 **최댓값 풀링(Max Pooling)** 과 **평균값 풀링(Average Pooling)** 이 있다.

- 최댓값 풀링: 특정 크기의 필터 내 원소값 중 가장 큰 값을 선택해 특징 맵의 크기를 감소시킨다.
- 평균값 풀링: 필터 내 원소값의 평균값으로 특징 맵의 크기를 감소시킨다.

합성곱 모델 구성

합성곱 신경망 모델은 입력 이미지 → 합성곱 계층, ReLU, 최댓값 풀링을 반복하고 3차원 배열을 1차원 벡터로 펼치는 flatten 연산 후 완전 연결 계층으로 전달해 출력 벡터를 반환한다.

```
# 예제 6.31 합성곱 모델
import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
            ),
            ...
        )
```

10개의 출력 벡터에 소프트맥스나 시그모이드를 적용하면 이미지를 입력했을 때 클래스를 분류하는 모델로 구축할 수 있다.

합성곱 신경망은 이미지와 유사한 구조를 가진 2차원 데이터에 더 적합하므로, 텍스트 데이터를 합성곱

신경망으로 처리한다면 각 단어를 지역적인 특징으로 인식할 수 있어 문장 내부의 구조적인 정보를 잘 파악할 수 있다. 또한 합성곱 신경망은 계산 속도가 빠르기 때문에 대규모 데이터셋에서도 빠르게 학습할 수 있다.



goblurry



이전 포스트

10. 비전 트랜스포머

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate